

Лабораторная работа №1

Generated by Doxygen 1.17.0

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 FullName Struct Reference	5
3.1.1 Detailed Description	5
3.1.2 Member Function Documentation	6
3.1.2.1 operator!=(())	6
3.1.2.2 operator<()	6
3.1.2.3 operator<=()	6
3.1.2.4 operator==(())	7
3.1.2.5 operator>()	7
3.1.2.6 operator>=()	8
3.2 Sportsman Struct Reference	8
3.2.1 Detailed Description	9
3.2.2 Constructor & Destructor Documentation	9
3.2.2.1 Sportsman()	9
3.2.3 Member Function Documentation	10
3.2.3.1 operator!=(())	10
3.2.3.2 operator<()	10
3.2.3.3 operator<=()	10
3.2.3.4 operator==(())	11
3.2.3.5 operator>()	11
3.2.3.6 operator>=()	12
3.2.3.7 to_csv()	12
3.2.4 Member Data Documentation	12
3.2.4.1 age	12
3.2.4.2 full_name	12
3.2.4.3 height_cm	13
3.2.4.4 sport	13
3.2.4.5 weight_kg	13
4 File Documentation	15
4.1 src/sorting.h File Reference	15
4.1.1 Detailed Description	15
4.1.2 Function Documentation	16
4.1.2.1 heap_sort()	16
4.1.2.2 insertion_sort()	16
4.1.2.3 quick_sort()	16

4.2 sorting.h	16
4.3 src/sportsman.h File Reference	17
4.3.1 Detailed Description	17
4.4 sportsman.h	17
4.5 src/utls.h File Reference	19
4.5.1 Detailed Description	20
4.5.2 Function Documentation	20
4.5.2.1 from_csv_file()	20
4.5.2.2 measure_time_ms()	20
4.5.2.3 to_csv_file()	21
4.6 utls.h	21
Index	23

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

FullName	ФИО спортсмена	5
Sportsman	Структура данных, описывающая спортсмена	8

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

src/sorting.h	Объявления алгоритмов сортировки	15
src/sportsman.h	Структура данных, описывающая спортсмена	17
src/utils.h	Вспомогательные функции	19

Chapter 3

Class Documentation

3.1 FullName Struct Reference

ФИО спортсмена

```
#include <sportsman.h>
```

Public Member Functions

- bool `operator==` (const `FullName` &other) const
Проверяет равенство двух ФИО
- bool `operator<` (const `FullName` &other) const
Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).
- bool `operator!=` (const `FullName` &other) const
Проверяет неравенство двух ФИО
- bool `operator>` (const `FullName` &other) const
Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).
- bool `operator<=` (const `FullName` &other) const
Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).
- bool `operator>=` (const `FullName` &other) const
Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).

Public Attributes

- `std::string last_name`
- `std::string first_name`
- `std::string middle_name`

3.1.1 Detailed Description

ФИО спортсмена

3.1.2 Member Function Documentation

3.1.2.1 operator!=(())

```
bool FullName::operator!=(  
    const FullName & other) const [inline]
```

Проверяет неравенство двух ФИО

Parameters

↔	ФИО для сравнения
[other]↔	

Returns

true, если ФИО не совпадают, иначе false

3.1.2.2 operator<()

```
bool FullName::operator<(  
    const FullName & other) const [inline]
```

Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).

Parameters

↔	ФИО для сравнения
[other]↔	

Returns

true, если текущее ФИО должно идти раньше другого, иначе false

3.1.2.3 operator<=()

```
bool FullName::operator<= (  
    const FullName & other) const [inline]
```

Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).

Parameters

↔ [other]↔	ФИО для сравнения
---------------	-------------------

Returns

true, если текущее ФИО должно идти не позже другого, иначе false

3.1.2.4 operator==()

```
bool FullName::operator== (  
    const FullName & other) const [inline]
```

Проверяет равенство двух ФИО

Parameters

↔ [other]↔	ФИО для сравнения
---------------	-------------------

Returns

true, если ФИО совпадают, иначе false

3.1.2.5 operator>()

```
bool FullName::operator> (  
    const FullName & other) const [inline]
```

Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).

Parameters

↔ [other]↔	ФИО для сравнения
---------------	-------------------

Returns

true, если текущее ФИО должно идти позже другого, иначе false

3.1.2.6 operator>=()

```
bool FullName::operator>= (
    const FullName & other) const [inline]
```

Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).

Parameters

↔	ФИО для сравнения
[other]↔	

Returns

true, если текущее ФИО должно идти не раньше другого, иначе false

The documentation for this struct was generated from the following file:

- [src/sportsman.h](#)

3.2 Sportsman Struct Reference

Структура данных, описывающая спортсмена

```
#include <sportsman.h>
```

Public Member Functions

- [Sportsman](#) (std::string csv_string)
Конструктор, который инициализирует поля структуры на основе строки в формате CSV (вид спорта, фамилия, имя, отчество, возраст, рост, вес).
- std::string [to_csv](#) () const
Вывод в строку в формате CSV (вид спорта, фамилия, имя, отчество, возраст, рост, вес).
- bool [operator==](#) (const [Sportsman](#) &other) const
Проверяет равенство двух спортсменов (сравнение по полям – вид спорта, ФИО, возраст).
- bool [operator<](#) (const [Sportsman](#) &other) const
Сравнивает двух спортсменов для сортировки (сравнение по полям – вид спорта, ФИО, возраст).
- bool [operator!=](#) (const [Sportsman](#) &other) const
Проверяет неравенство двух спортсменов (сравнение по полям – вид спорта, ФИО, возраст).
- bool [operator>](#) (const [Sportsman](#) &other) const
Сравнивает двух спортсменов для сортировки (сравнение по полям – вид спорта, ФИО, возраст).
- bool [operator<=](#) (const [Sportsman](#) &other) const
Сравнивает двух спортсменов для сортировки (сравнение по полям – вид спорта, ФИО, возраст).
- bool [operator>=](#) (const [Sportsman](#) &other) const
Сравнивает двух спортсменов для сортировки (сравнение по полям – вид спорта, ФИО, возраст).

Public Attributes

- [FullName](#) `full_name`
- unsigned short [age](#)
- unsigned short [height_cm](#)
- unsigned short [weight_kg](#)
- `std::string` [sport](#)

3.2.1 Detailed Description

Структура данных, описывающая спортсмена

3.2.2 Constructor & Destructor Documentation

3.2.2.1 Sportsman()

```
Sportsman::Sportsman (  
    std::string csv_string) [inline]
```

Конструктор, который инициализирует поля структуры на основе строки в формате CSV (вид спорта, фамилия, имя, отчество, возраст, рост, вес).

Parameters

<code>[csv_string]</code>	Строка в формате CSV
---------------------------	----------------------

Returns

Инициализированный объект структуры [Sportsman](#)

Exceptions

<code>std::invalid_argument</code> , если	строка не является корректной CSV строкой или содержит некорректные данные
---	--

3.2.3 Member Function Documentation

3.2.3.1 operator!=()

```
bool Sportsman::operator!= (
    const Sportsman & other) const [inline]
```

Проверяет неравенство двух спортсменов (сравнение по полям – вид спорта, ФИО, возраст).

Parameters

↔ [other]↔	Спортсмен для сравнения
---------------	-------------------------

Returns

true, если спортсмены не совпадают, иначе false

3.2.3.2 operator<()

```
bool Sportsman::operator< (
    const Sportsman & other) const [inline]
```

Сравнивает двух спортсменов для сортировки (сравнение по полям – вид спорта, ФИО, возраст).

Parameters

↔ [other]↔	Спортсмен для сравнения
---------------	-------------------------

Returns

true, если текущий спортсмен должен идти раньше другого, иначе false

3.2.3.3 operator<=()

```
bool Sportsman::operator<= (
    const Sportsman & other) const [inline]
```

Сравнивает двух спортсменов для сортировки (сравнение по полям – вид спорта, ФИО, возраст).

Parameters

$\leftrightarrow$ [other] $\leftrightarrow$	Спортсмен для сравнения
--	-------------------------

Returns

true, если текущий спортсмен должен идти не позже другого, иначе false

3.2.3.4 operator==()

```
bool Sportsman::operator== (  
    const Sportsman & other) const [inline]
```

Проверяет равенство двух спортсменов (сравнение по полям – вид спорта, ФИО, возраст).

Parameters

$\leftrightarrow$ [other] $\leftrightarrow$	Спортсмен для сравнения
--	-------------------------

Returns

true, если спортсмены совпадают, иначе false

3.2.3.5 operator>()

```
bool Sportsman::operator> (  
    const Sportsman & other) const [inline]
```

Сравнивает двух спортсменов для сортировки (сравнение по полям – вид спорта, ФИО, возраст).

Parameters

$\leftrightarrow$ [other] $\leftrightarrow$	Спортсмен для сравнения
--	-------------------------

Returns

true, если текущий спортсмен должен идти позже другого, иначе false

3.2.3.6 operator>=()

```
bool Sportsman::operator>= (
    const Sportsman & other) const [inline]
```

Сравнивает двух спортсменов для сортировки (сравнение по полям – вид спорта, ФИО, возраст).

Parameters

↔	Спортсмен для сравнения
[other]↔	

Returns

true, если текущий спортсмен должен идти не раньше другого, иначе false

3.2.3.7 to_csv()

```
std::string Sportsman::to_csv () const [inline]
```

Вывод в строку в формате CSV (вид спорта, фамилия, имя, отчество, возраст, рост, вес).

Returns

Строка в формате CSV

3.2.4 Member Data Documentation

3.2.4.1 age

```
unsigned short Sportsman::age
```

Возраст спортсмена в годах

3.2.4.2 full_name

```
FullName Sportsman::full_name
```

ФИО спортсмена

3.2.4.3 height_cm

```
unsigned short Sportsman::height_cm
```

Рост спортсмена в сантиметрах

3.2.4.4 sport

```
std::string Sportsman::sport
```

Спорт, которым занимается спортсмен

3.2.4.5 weight_kg

```
unsigned short Sportsman::weight_kg
```

Вес спортсмена в килограммах

The documentation for this struct was generated from the following file:

- [src/sportsman.h](#)

Chapter 4

File Documentation

4.1 src/sorting.h File Reference

Объявления алгоритмов сортировки

```
#include "sportsman.h"  
#include <vector>
```

Functions

- void [insertion_sort](#) (std::vector< [Sportsman](#) > &vec)
Сортировка простыми вставками
- void [heap_sort](#) (std::vector< [Sportsman](#) > &vec)
Пирамидальная сортировка
- void [quick_sort](#) (std::vector< [Sportsman](#) > &vec)
Быстрая сортировка

4.1.1 Detailed Description

Объявления алгоритмов сортировки

4.1.2 Function Documentation

4.1.2.1 heap_sort()

```
void heap_sort (
    std::vector< Sportsman > & vec)
```

Пирамидальная сортировка

Parameters

vec	Ссылка на вектор для сортировки
-----	---------------------------------

4.1.2.2 insertion_sort()

```
void insertion_sort (
    std::vector< Sportsman > & vec)
```

Сортировка простыми вставками

Parameters

vec	Ссылка на вектор для сортировки
-----	---------------------------------

4.1.2.3 quick_sort()

```
void quick_sort (
    std::vector< Sportsman > & vec)
```

Быстрая сортировка

Parameters

vec	Ссылка на вектор для сортировки
-----	---------------------------------

4.2 sorting.h

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include "sportsman.h"
00009 #include <vector>
00010
00015 void insertion_sort(std::vector<Sportsman>& vec);
00016
00021 void heap_sort(std::vector<Sportsman>& vec);
00022
00027 void quick_sort(std::vector<Sportsman>& vec);
```

4.3 src/sportsman.h File Reference

Структура данных, описывающая спортсмена

```
#include <stdexcept>
#include <string>
```

Classes

- struct [FullName](#)
ФИО спортсмена
- struct [Sportsman](#)
Структура данных, описывающая спортсмена

4.3.1 Detailed Description

Структура данных, описывающая спортсмена

4.4 sportsman.h

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include <stdexcept>
00009 #include <string>
00010
00015 struct FullName
00016 {
00017     std::string last_name;    /**< Фамилия */
00018     std::string first_name;   /**< Имя */
00019     std::string middle_name;  /**< Отчество */
00020
00026     bool operator==(const FullName &other) const
00027     {
00028         return first_name == other.first_name &&
00029                middle_name == other.middle_name &&
00030                last_name == other.last_name;
00031     }
00032
00038     bool operator<(const FullName &other) const
00039     {
00040         if (last_name != other.last_name)
00041         {
00042             return last_name < other.last_name;
00043         }
00044         if (first_name != other.first_name)
00045         {
00046             return first_name < other.first_name;
00047         }
00048         return middle_name < other.middle_name;
00049     }
00050
00056     bool operator!=(const FullName &other) const
00057     {
00058         return !(*this == other);
00059     }
00060
00066     bool operator>(const FullName &other) const
00067     {
```

```

00068         return other < *this;
00069     }
00070
00076     bool operator<=(const FullName &other) const
00077     {
00078         return !(other < *this);
00079     }
00080
00086     bool operator>=(const FullName &other) const
00087     {
00088         return !(*this < other);
00089     }
00090 };
00091
00096 struct Sportsman
00097 {
00098     FullName full_name;
00099     unsigned short age;
00100     unsigned short height_cm;
00101     unsigned short weight_kg;
00102     std::string sport;
00103
00110     Sportsman(std::string csv_string)
00111     {
00112         size_t pos = 0;
00113         std::string token;
00114         std::string delimiter = ",";
00115         int field_index = 0;
00116
00117         for (int field_idx = 0; field_idx < 7; field_idx++)
00118         {
00119             pos = csv_string.find(delimiter);
00120             if (pos == std::string::npos)
00121             {
00122                 if (field_idx != 6)
00123                 {
00124                     throw std::invalid_argument("Недостаточно полей в CSV строке");
00125                 }
00126                 token = csv_string;
00127             }
00128             else if (field_idx == 6)
00129             {
00130                 throw std::invalid_argument("Слишком много полей в CSV строке");
00131             }
00132             else
00133             {
00134                 token = csv_string.substr(0, pos);
00135                 csv_string.erase(0, pos + delimiter.length());
00136             }
00137
00138             switch (field_idx)
00139             {
00140                 case 0:
00141                     sport = token;
00142                     break;
00143                 case 1:
00144                     full_name.last_name = token;
00145                     break;
00146                 case 2:
00147                     full_name.first_name = token;
00148                     break;
00149                 case 3:
00150                     full_name.middle_name = token;
00151                     break;
00152                 case 4:
00153                     age = (unsigned short)std::stoul(token);
00154                     break;
00155                 case 5:
00156                     height_cm = (unsigned short)std::stoul(token);
00157                     break;
00158                 case 6:
00159                     weight_kg = (unsigned short)std::stoul(token);
00160                     break;
00161             }
00162         }
00163     }
00164
00169     std::string to_csv() const
00170     {
00171         return sport + "," +
00172             full_name.last_name + "," +
00173             full_name.first_name + "," +
00174             full_name.middle_name + "," +
00175             std::to_string(age) + "," +
00176             std::to_string(height_cm) + "," +
00177             std::to_string(weight_kg);
00178     }

```

```

00179
00185     bool operator==(const Sportsman &other) const
00186     {
00187         return sport == other.sport &&
00188             full_name == other.full_name &&
00189             age == other.age;
00190     }
00191
00197     bool operator<(const Sportsman &other) const
00198     {
00199         if (sport != other.sport)
00200         {
00201             return sport < other.sport;
00202         }
00203         if (full_name != other.full_name)
00204         {
00205             return full_name < other.full_name;
00206         }
00207         return age < other.age;
00208     }
00209
00215     bool operator!=(const Sportsman &other) const
00216     {
00217         return !(*this == other);
00218     }
00219
00225     bool operator>(const Sportsman &other) const
00226     {
00227         return other < *this;
00228     }
00229
00235     bool operator<=(const Sportsman &other) const
00236     {
00237         return !(other < *this);
00238     }
00239
00245     bool operator>=(const Sportsman &other) const
00246     {
00247         return !(*this < other);
00248     }
00249 };

```

4.5 src/utils.h File Reference

Вспомогательные функции

```

#include "sportsman.h"
#include <chrono>
#include <functional>
#include <iostream>
#include <string>
#include <vector>

```

Functions

- `std::vector< Sportsman > from_csv_file` (const std::string &filename)
Прочитать вектор спортсменов из CSV файла
- `void to_csv_file` (const std::vector< Sportsman > &data, const std::string &filename)
Записать вектор спортсменов в CSV-файл
- `double measure_time_ms` (std::function< void(std::vector< Sportsman > &)> sort_func, const std::vector< Sportsman > &source)
Измерение времени сортировки

4.5.1 Detailed Description

Вспомогательные функции

4.5.2 Function Documentation

4.5.2.1 from_csv_file()

```
std::vector< Sportsman > from_csv_file (  
    const std::string & filename)
```

Прочитать вектор спортсменов из CSV файла

Parameters

filename	Путь к входному файлу
----------	-----------------------

Returns

Вектор [Sportsman](#)

4.5.2.2 measure_time_ms()

```
double measure_time_ms (  
    std::function< void(std::vector< Sportsman > &)> sort_func,  
    const std::vector< Sportsman > & source)
```

Измерение времени сортировки

Parameters

sort_func	Функция сортировки (void(std::vector<Sportsman>&))
source	Вектор спортсменов для сортировки

Returns

Время сортировки (в миллисекундах)

4.5.2.3 to_csv_file()

```
void to_csv_file (
    const std::vector< Sportsman > & data,
    const std::string & filename)
```

Записать вектор спортсменов в CSV-файл

Parameters

data	Вектор Sportsman
filename	Путь к выходному файлу

4.6 utils.h

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include "sportsman.h"
00009 #include <chrono>
00010 #include <functional>
00011 #include <iostream>
00012 #include <string>
00013 #include <vector>
00014
00020 std::vector<Sportsman> from_csv_file(const std::string &filename);
00021
00027 void to_csv_file(const std::vector<Sportsman> &data, const std::string &filename);
00028
00035 double measure_time_ms(std::function<void(std::vector<Sportsman> &)> sort_func, const
    std::vector<Sportsman> &source);
```


Index

- age
 - Sportsman, [12](#)
- from_csv_file
 - utils.h, [20](#)
- full_name
 - Sportsman, [12](#)
- FullName, [5](#)
 - operator!=, [6](#)
 - operator<, [6](#)
 - operator<=, [6](#)
 - operator>, [7](#)
 - operator>=, [7](#)
 - operator==, [7](#)
- heap_sort
 - sorting.h, [16](#)
- height_cm
 - Sportsman, [12](#)
- insertion_sort
 - sorting.h, [16](#)
- measure_time_ms
 - utils.h, [20](#)
- operator!=
 - FullName, [6](#)
 - Sportsman, [10](#)
- operator<
 - FullName, [6](#)
 - Sportsman, [10](#)
- operator<=
 - FullName, [6](#)
 - Sportsman, [10](#)
- operator>
 - FullName, [7](#)
 - Sportsman, [11](#)
- operator>=
 - FullName, [7](#)
 - Sportsman, [11](#)
- operator==
 - FullName, [7](#)
 - Sportsman, [11](#)
- quick_sort
 - sorting.h, [16](#)
- sorting.h
 - heap_sort, [16](#)
 - insertion_sort, [16](#)
 - quick_sort, [16](#)
 - sport
 - Sportsman, [13](#)
 - Sportsman, [8](#)
 - age, [12](#)
 - full_name, [12](#)
 - height_cm, [12](#)
 - operator!=, [10](#)
 - operator<, [10](#)
 - operator<=, [10](#)
 - operator>, [11](#)
 - operator>=, [11](#)
 - operator==, [11](#)
 - sport, [13](#)
 - Sportsman, [9](#)
 - to_csv, [12](#)
 - weight_kg, [13](#)
 - src/sorting.h, [15](#), [16](#)
 - src/sportsman.h, [17](#)
 - src/utils.h, [19](#), [21](#)
- to_csv
 - Sportsman, [12](#)
- to_csv_file
 - utils.h, [20](#)
- utils.h
 - from_csv_file, [20](#)
 - measure_time_ms, [20](#)
 - to_csv_file, [20](#)
- weight_kg
 - Sportsman, [13](#)